

Artificial Intelligence

Lecture 12, Chapter 18 Learning from Examples

Supta Richard Philip

<https://suptaphilip.github.io/>

Department of CS
AIUB

AIUB, June 2026



Table of Contents

- 1 Forms of Learning
- 2 Supervised Learning
- 3 Regression and Classification with Linear Models
 - Univariate linear regression
 - Multivariate linear regression
 - Linear classifiers with a hard threshold
 - Linear classification with logistic regression
- 4 Artificial Neural Networks
 - Artificial Neural Structure
 - Single-layer feed-forward neural networks (perceptrons)
 - Multilayer feed-forward neural networks
 - Learning in multilayer networks
- 5 References



Table of Contents

- 1 Forms of Learning
- 2 Supervised Learning
- 3 Regression and Classification with Linear Models
 - Univariate linear regression
 - Multivariate linear regression
 - Linear classifiers with a hard threshold
 - Linear classification with logistic regression
- 4 Artificial Neural Networks
 - Artificial Neural Structure
 - Single-layer feed-forward neural networks (perceptrons)
 - Multilayer feed-forward neural networks
 - Learning in multilayer networks
- 5 References



Learning

- An **agent is learning** if it **improves its performance** on future tasks after making **observations** about the world.
- In this chapter we will concentrate on one class of learning problem, which seems restricted but actually has vast applicability: from **a collection of input–output pairs, learn a function that predicts the output for new inputs.**
- We say that learning a (possibly incorrect) general function or rule from specific input–output pairs is called **inductive learning.**



Feedback to learn from

There are three types of feedback that determine the three main types of learning:

- In **unsupervised learning** the agent learns patterns in the input even though no explicit feedback is supplied. The most common unsupervised learning task is **clustering**: detecting potentially useful clusters of input examples.
- In **reinforcement learning** the agent learns from a series of reinforcements—rewards or punishments.
- In **supervised learning** the agent observes some example input–output pairs and learns a function that maps from input to output.
- In **semi-supervised learning** we are given a few labeled examples and must make what we can of a large collection of unlabeled examples.



Table of Contents

- 1 Forms of Learning
- 2 Supervised Learning
- 3 Regression and Classification with Linear Models
 - Univariate linear regression
 - Multivariate linear regression
 - Linear classifiers with a hard threshold
 - Linear classification with logistic regression
- 4 Artificial Neural Networks
 - Artificial Neural Structure
 - Single-layer feed-forward neural networks (perceptrons)
 - Multilayer feed-forward neural networks
 - Learning in multilayer networks
- 5 References



Supervised Learning

The task of supervised learning is this:

- Given a training set of N example input–output pairs $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$, where each y_j was generated by an unknown function $y = f(x)$, discover a function h that approximates the true function f .
- Here x and y can be any value; they need not be numbers. The function h is a hypothesis.
- **Learning is a search** through the space of possible hypotheses for one that will perform well, even on new examples beyond the training set.



Supervised Learning

- To measure the **accuracy of a hypothesis** we give it a **test set** of examples that are distinct from the training set.
- We say a hypothesis **generalizes well** if it correctly predicts the value of y for novel examples.
- When the **output y** is one of a finite set of values (such as sunny, cloudy or rainy), the learning problem is called **classification**, and is called Boolean or **binary classification** if there are only two values.
- When y is a **number** (such as tomorrow's temperature), the learning problem is called **regression**.



Supervised Learning

- Example shows that fitting a function of a single variable to some data points. The examples are points in the (x, y) plane, where $y = f(x)$. We don't know what f is, but we will approximate it with a function h selected from a hypothesis space, H , which for this example we will take to be the set of polynomials.

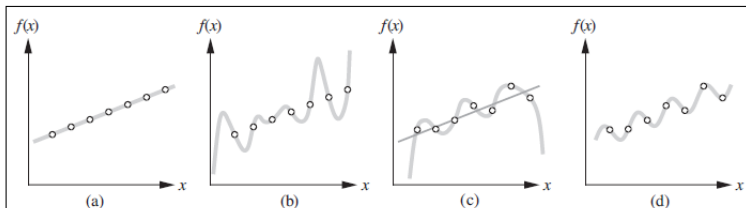


Figure 18.1 (a) Example $(x, f(x))$ pairs and a consistent, linear hypothesis. (b) A consistent, degree-7 polynomial hypothesis for the same data set. (c) A different data set, which admits an exact degree-6 polynomial fit or an approximate linear fit. (d) A simple, exact sinusoidal fit to the same data set.



Supervised Learning

- Figure 18.1(a) shows some data with an exact fit by a straight line (the polynomial $0.4x + 3$). The line is called a **consistent hypothesis** because it agrees with all the data.
- Figure 18.1(b) shows a high degree polynomial that is also consistent with the same data.
- This illustrates a fundamental problem in inductive learning: how do we choose from among multiple consistent hypotheses? One answer is to prefer the **simplest hypothesis consistent with the data**. This principle is called **Ockham's razor**.
- Figure 18.1(c) shows a second data set. There is no consistent straight line for this data set; in fact, it requires a degree-6 polynomial for an exact fit.



Supervised Learning

- In Figure 18.1(d) we expand the hypothesis space H to allow polynomials over both x and $\sin(x)$, and find that the data in (c) can be fitted exactly by a simple function of the form $ax + b + c\sin(x)$.
- In general, there is a **tradeoff** between **complex hypotheses that fit the training data well** and **simpler hypotheses that may generalize better**.



Table of Contents

- 1 Forms of Learning
- 2 Supervised Learning
- 3 Regression and Classification with Linear Models
 - Univariate linear regression
 - Multivariate linear regression
 - Linear classifiers with a hard threshold
 - Linear classification with logistic regression
- 4 Artificial Neural Networks
 - Artificial Neural Structure
 - Single-layer feed-forward neural networks (perceptrons)
 - Multilayer feed-forward neural networks
 - Learning in multilayer networks
- 5 References



Univariate linear regression

- A **univariate linear function** (a straight line) with **input x** and **output y** has the form $y = w_1x + w_0$, where w_0 and w_1 are real-valued coefficients to be learned.
- We use the letter w because we think of the **coefficients as weights**; the value of y is changed by changing the relative weight of one term or another.
- We'll define w to be the **vector $[w_0, w_1]$** , and define $h_w(x) = w_1x + w_0$.
- The task of finding the h_w that best fits these data is called **linear regression**.



Univariate linear regression

- Figure shows an example of a training set of n points in the x, y plane, each point representing the size in square feet and the price of a house offered for sale.

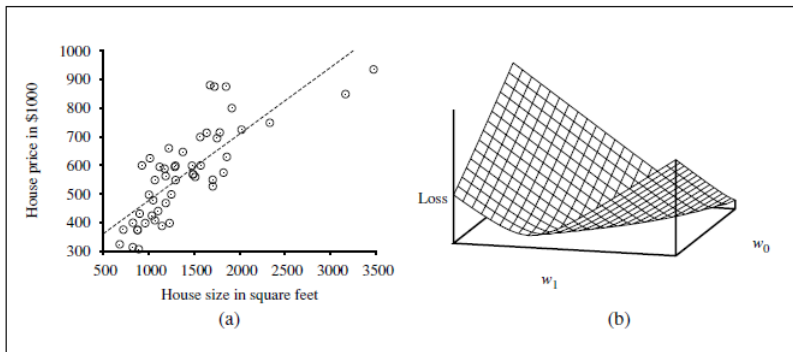


Figure 18.13 (a) Data points of price versus floor space of houses for sale in Berkeley, CA, in July 2009, along with the linear function hypothesis that minimizes squared error loss: $y = 0.232x + 246$. (b) Plot of the loss function $\sum_j (w_1 x_j + w_0 - y_j)^2$ for various values of w_0, w_1 . Note that the loss function is convex, with a single global minimum.

Univariate linear regression

- To fit a line to the data, all we have to do is find the values of the weights $[w_0, w_1]$ that minimize the empirical loss. It is traditional to use the squared loss function, L_2 , summed over all the training examples:

$$\text{Loss}(h_w) = \sum_{j=1}^N L_2(y_j, h_w(x_j)) = \sum_{j=1}^N (y_j - h_w(x_j))^2 = \sum_{j=1}^N (y_j - (w_1 x_j + w_0))^2$$

- We would like to find $w^* = \text{argmin}_w \text{Loss}(h_w)$.
- The sum $\sum_{j=1}^N (y_j - (w_1 x_j + w_0))^2$ minimized when its partial derivatives with respect to w_0 and w_1 are zero:

$$\frac{\partial}{\partial w_0} \sum_{j=1}^N (y_j - (w_1 x_j + w_0))^2 = 0$$

$$\frac{\partial}{\partial w_1} \sum_{j=1}^N (y_j - (w_1 x_j + w_0))^2 = 0$$



Univariate linear regression

- These equations have a unique solution:

$$w_1 = \frac{N \sum (x_j y_j) (\sum x_j) (\sum y_j)}{N (\sum x_j^2) (\sum x_j)^2}; w_0 = \frac{(\sum y_j - w_1 (\sum x_j))}{N}$$

- For the example in Figure, the solution is $w_1 = 0.232$, $w_0 = 246$, and the line with those weights is shown as a dashed line in the figure.
- Many forms of learning involve adjusting weights to minimize a loss, so it helps to have a mental picture of what's going on in weight space-the space defined by all possible settings of the weights.
- For univariate linear regression, the weight space defined by w_0 and w_1 is two-dimensional, so we can plot graph the loss as a function of w_0 and w_1 in a 3D plot.



Univariate linear regression

- We see that the **loss function is convex**; this is true for every linear regression problem with an L_2 loss function, and implies that there are no local minima.
- In some sense that's the end of the story for linear models; if we need to fit lines to data, we apply closed form Equation.
- To go beyond linear models, we will need to face the fact that the equations defining minimum loss (as in Equation) will often have no closed-form solution.
- Instead, we will face a general optimization search problem in a continuous weight space.
- such problems can be addressed by a hill-climbing algorithm that follows the gradient of the function to be optimized.



Gradient Descent

- In this case, because we are trying to minimize the loss, we will use **gradient descent**.
- We choose any starting point in weight space—here, a point in the (w_0, w_1) plane—and then move to a neighboring point that is downhill, repeating until we converge on the minimum possible loss.

w $\leftarrow$ any point in the parameter space

loop until convergence do

for each w_i in w do

$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Loss}(\mathbf{w})$$

- The parameter α , which we called the step size, is usually called the **learning rate**.



Gradient Descent

- Let's first work out the partial derivatives—the slopes—in the simplified case of only one training example, (x, y) :

$$\begin{aligned}\frac{\partial}{\partial w_i} \text{Loss}(w) &= \frac{\partial}{\partial w_i} (y - h_w(x))^2 = \\ 2(y - h_w(x)) * \frac{\partial}{\partial w_i} (y - h_w(x)) &= 2(y - h_w(x)) * \frac{\partial}{\partial w_i} (y - (w_1 x + w_0))\end{aligned}$$

- applying this to both w_0 and w_1 we get:

$$\begin{aligned}\frac{\partial}{\partial w_0} \text{Loss}(W) &= -2(y - h_w(x)) \\ \frac{\partial}{\partial w_1} \text{Loss}(W) &= -2(y - h_w(x)) * x\end{aligned}$$

- Then, plugging this back into Equation, and folding the 2 into the unspecified learning rate α , we get the following learning rule for the weights:

$$\begin{aligned}w_0 &\leftarrow w_0 + \alpha(y - h_w(x)) \\ w_1 &\leftarrow w_1 + \alpha(y - h_w(x)) * x\end{aligned}$$



Batch gradient descent

- For N training examples, we want to minimize the sum of the individual losses for each example. The derivative of a sum is the sum of the derivatives, so we have:

$$w_0 \leftarrow w_0 + \alpha \sum_j (y_j - h_w(x_j))$$

$$w_1 \leftarrow w_1 + \alpha \sum_j (y_j - h_w(x_j)) * x_j$$

- These updates constitute the **batch gradient descent learning rule** for univariate linear regression.
- Convergence to the unique global minimum is guaranteed (as long as we pick α small enough) but may be very slow: we have to cycle through all the training data for every step, and there may be many steps.
- There is another possibility, called **stochastic gradient descent**, where we consider only a single training point at a time, taking a step after each one using weight update Equation.



Multivariate linear regression

- We can easily extend to multivariate linear regression problems, in which each example x_j is an n -element vector.
- Our hypothesis space is the set of functions of the form
$$h_{sw}(x_j) = w_0 + w_1x_{j,1} + \dots + w_nx_{j,n} = w_0 + \sum_i w_ix_{j,i}.$$
- The w_0 term, the intercept, stands out as different from the others. We can fix that by inventing a dummy input attribute, $x_{j,0}$, which is defined as always equal to 1.
- Then h is simply the dot product of the weights and the input vector.
$$h_{sw}(x_j) = w \cdot x_j = w^T x_j = \sum_i w_ix_{j,i}.$$
- The best vector of weights, w^* , minimizes squared-error loss over the examples:

$$w^* = \operatorname{argmin}_w \sum_j L_2(y_j, w \cdot x_j)$$



Multivariate linear regression

- Multivariate linear regression is actually not much more complicated than the univariate case.
- Gradient descent will reach the (unique) minimum of the loss function; the update equation for each weight w_i is
$$w_i \leftarrow w_i + \alpha \sum_j x_{j,i}(y_j - h_w(x_j))$$



Gradient Descent Numerical Example

$$\hat{y} = \theta_0 + \theta_1 x$$

$$\theta_0 = 0.1 \quad \theta_1 = 0.1$$

x	y	$\hat{y} = \theta_0 + \theta_1 x$	SSE $\frac{1}{2}(\hat{y} - y)^2$	$\frac{\partial(SSE)}{\partial \theta_0}$ $\hat{y} - y$	$\frac{\partial(SSE)}{\partial \theta_1}$ $(\hat{y} - y)x$
0.2	0.44	0.12	0.0512	-0.32	-0.064
0.31	0.123	0.131	0.000032	0.008	0.00248
0.45	0.75	0.145	0.183	-0.605	-0.27225
0.26	0.39	0.175	0.0231	-0.215	-0.16125
				-1.132	-0.495

$$\theta_0 := \theta_0 - \alpha \sum_{i=1}^n \frac{\partial(SSE)}{\partial \theta_0}$$

$$\theta_0 = 0.1 - 0.01 \times (-1.132)$$

$$\theta_0 = 0.11132$$

$$\theta_1 := \theta_1 - \alpha \sum_{i=1}^n \frac{\partial(SSE)}{\partial \theta_1}$$

$$\theta_1 = 0.1 - 0.01 \times (-0.495)$$

$$\theta_1 = 0.10495$$

$$\alpha = 0.01$$



Exercise

- Optimization using Gradient descent $w^* = \operatorname{argmin}_w \operatorname{Loss}(h_w)$.

Loss Function(squared loss L_2)

$$\sum_{j=1}^N L_2(y_j, h_w(x_j)) = \sum_{j=1}^N (y_j - h_w(x_j))^2 \text{ where}$$

$$h_w(x) = w_0 + w_1x_1 + w_2x_2.$$

Show the derivation of the Gradient Descent update rules for a linear model with two features x_1 and x_2 .

- Derive the parameter update equations for the Gradient Descent algorithm when the model contains three input features x_1 , x_2 and x_3 .
- Derive the update rules for w_0 , w_1 and w_2 parameters where general updates rule is $w_i \leftarrow w_i + \alpha \sum_j x_{j,i}(y_j - h_w(x_j))$ and α is small constant(learning rate).



Exercise: Multivariate linear regression using Gradient descent Algorithm

Show one iteration of Optimization using Gradient descent algorithm $w^* = \operatorname{argmin}_w \operatorname{Loss}(h_w)$. hypothesis space is the set of linear functions $h_w(x) = w_0 + w_1x_1 + w_2x_2$.

Loss Function(squared loss L_2) $\sum_{j=1}^m (y_j - h_w(x_j))^2$ where m is number of examples and w_0 , w_1 and w_2 are parameters.

initial value of $w_0 = 1$, $w_1 = 1.5$ and $w_2 = 2$ and learning rate, $\alpha = 0.01$.

x_1	x_2	y
21	2	40
16	3	33
12	4	25
7	3	18

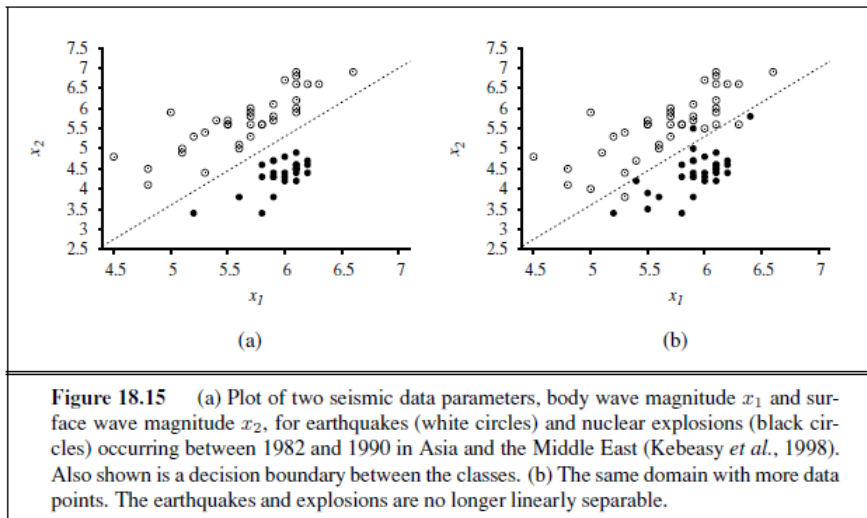


Linear classifiers with a hard threshold

- Linear functions can be used to do classification as well as regression.
- For example, Figure shows data points of two classes: earthquakes and underground explosions.
- Each point is defined by two input values, x_1 and x_2 , that refer to body and surface wave magnitudes computed from the seismic signal.
- Given these training data, the task of classification is to learn a hypothesis h that will take new (x_1, x_2) points and return either 0 for earthquakes or 1 for explosions.



Linear classifiers with a hard threshold



Linear classifiers with a hard threshold

- A decision boundary is a line (or a surface, in higher dimensions) that separates the two classes.
- In Figure, the decision boundary is a straight line.
- A linear decision boundary is called a linear separator and data that admit such a separator are called linearly separable.
- we can think of h as the result of passing the linear function $w \cdot x$ through a threshold function:
 $h_w(x) = \text{Threshold}(w \cdot x)$ where $\text{Threshold}(z) = 1$ if $z \geq 0$ and 0 otherwise.

$$h_w(x) = \begin{cases} 1 & \text{if } (w_i \cdot x_i + w_0) \geq 0 \\ 0 & \text{otherwise} \end{cases}$$



Linear classifiers with a hard threshold

- There is a simple weight update rule that converges to a solution—that is, a linear separator that classifies the data perfectly if provided the data are linearly separable.
- For a single example (x, y) , we have
$$w_i \leftarrow w_i + \alpha(y - h_w(x)) * x_i$$
and α is small constant(learning rate).
- This rule is called **the perceptron learning rule**.
- Because we are considering a 0/1 classification problem. Both the true value y and the hypothesis output $h_w(x)$ are either 0 or 1.



Perceptron learning rule

- If the output is correct, i.e., $y = h_w(x)$, then the weights are not changed.
- If y is 1 but $h_w(x)$ is 0, then w_i is increased when the corresponding input x_i is positive and decreased when x_i is negative. This makes sense, because we want to make $w \cdot x$ bigger so that $h_w(x)$ outputs a 1.
- If y is 0 but $h_w(x)$ is 1, then w_i is decreased when the corresponding input x_i is positive and increased when x_i is negative. This makes sense, because we want to make $w \cdot x$ smaller so that $h_w(x)$ outputs a 0.



Linear classification with logistic regression

- We have seen that passing the output of a linear function through the threshold function creates a linear classifier but yet the hard nature of the threshold causes some problems[**hard threshold**].
- The hypothesis $h_w(x)$ is not differentiable and is in fact a discontinuous function of its inputs and its weights.

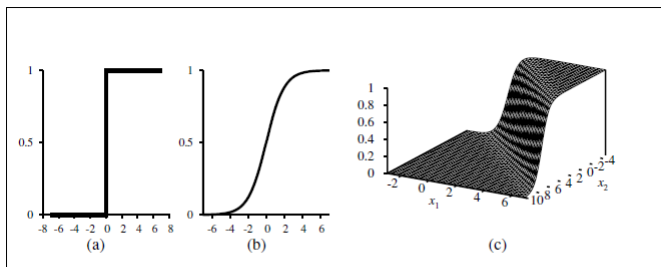


Figure 18.17 (a) The hard threshold function $\text{Threshold}(z)$ with 0/1 output. Note that the function is nondifferentiable at $z=0$. (b) The logistic function, $\text{Logistic}(z) = \frac{1}{1+e^{-z}}$, also known as the sigmoid function. (c) Plot of a logistic regression hypothesis $h_w(x) = \text{Logistic}(w \cdot x)$ for the data shown in Figure 18.15(b).



Linear classification with logistic regression

- Furthermore, the linear classifier always announces a completely confident prediction of 1 or 0, even for examples that are very close to the boundary.
- All of these issues can be resolved to a large extent by softening the threshold function-approximating the hard threshold with a **continuous, differentiable function**.

- **Soft thresholds: the logistic function.**

$$\text{Logistic}(z) = \frac{1}{1+e^{-z}}$$

- With the logistic function replacing the threshold function, we now have $h_w(x) = \text{Logistic}(w \cdot x) = \frac{1}{1+e^{-(w \cdot x + w_0)}}$.



Linear classification with logistic regression

- The process of fitting the weights of this model to minimize loss on a data set is called logistic regression.
- There is no easy closed-form solution to find the optimal value of w with this model, but the gradient descent computation is straightforward.
- Because our hypotheses no longer output just 0 or 1, we will use the L_2 loss function; also, to keep the formulas readable, we'll use g to stand for the logistic function, with g' its derivative.
- For a single example (x, y) , the derivation of the gradient is the same as for linear regression up to the point where the actual form of h is inserted.



Linear classification with logistic regression

- For this derivation, we will need the chain rule:

$$\frac{\partial g(f(x))}{\partial x} = g'(f(x)) \frac{\partial f(x)}{\partial x}$$

- $$\begin{aligned}\frac{\partial}{\partial w_i} \text{Loss}(w) &= \frac{\partial}{\partial w_i} (y - h_w(x))^2 = 2(y - h_w(x)) * \frac{\partial}{\partial w_i} (y - h_w(x)) \\ &= -2(y - h_w(x)) * g'(w \cdot x) * \frac{\partial w \cdot x}{\partial w_i} \\ &= -2(y - h_w(x)) * g'(w \cdot x) * x_i\end{aligned}$$

- The derivative g' of the logistic function satisfies

$$g'(z) = g(z)(1 - g(z)).$$

- $$g(w \cdot x) = g(w \cdot x)(1 - g(w \cdot x)) = h_w(x)(1 - h_w(x))$$

- so the weight update for minimizing the loss is

$$w_i \leftarrow w_i + \alpha (y - h_w(x)) * h_w(x)(1 - h_w(x)) * x_i$$



Exercise from Linear classifiers with a hard threshold

Show weights update where initial value of $w_0 = 1$, $w_1 = 1.5$ and $w_2 = 2$ and learning rate, $\alpha = 0.01$.

$$h_w(x) = \begin{cases} 1 & \text{if } (w_i \cdot x_i + w_0) \geq 0 \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

For a single example (x, y) , we have the perceptron learning rule
 $w_i \leftarrow w_i + \alpha(y - h_w(x)) * x_i$
and α is small constant(learning rate).

x_1	x_2	y
8	2	0
16	3	1
12	4	1
7	3	0



Exercise from Linear classification with logistic regression

Loss Function(squared loss L_2) $\sum_{j=1}^m (y_j - h_w(x_j))^2$ where m is number of examples and w_0 , w_1 and w_2 are parameters.

We have now logistic function in stead of hard threshold function,

$$h_w(x) = \text{Logistic}(w \cdot x) = \frac{1}{1 + e^{-(w \cdot x + w_0)}}$$

For a single example (x, y) , the weight update for minimizing the loss is

$$w_i \leftarrow w_i + \alpha(y - h_w(x)) * h_w(x)(1 - h_w(x)) * x_i$$

x_1	x_2	y
8	2	0
16	3	1
12	4	1
7	3	0

Show weight update where initial value of $w_0 = 1$, $w_1 = 1.5$ and $w_2 = 2$ and learning rate, $\alpha = 0.01$.



Table of Contents

- 1 Forms of Learning
- 2 Supervised Learning
- 3 Regression and Classification with Linear Models
 - Univariate linear regression
 - Multivariate linear regression
 - Linear classifiers with a hard threshold
 - Linear classification with logistic regression
- 4 Artificial Neural Networks
 - Artificial Neural Structure
 - Single-layer feed-forward neural networks (perceptrons)
 - Multilayer feed-forward neural networks
 - Learning in multilayer networks
- 5 References



Artificial Neural Networks

- We turn now to what seems to be a somewhat unrelated topic: the **brain**.
- the technical ideas we have discussed so far turn out to be useful in building mathematical models of the brain's activity.
- The hypothesis that mental activity consists primarily of electrochemical activity in networks of brain cells called **neurons**.
- Inspired by this hypothesis, some of the earliest AI work aimed to create artificial neural networks.
- Figure shows a simple mathematical model of the neuron devised by McCulloch and Pitts(1943).
- It “fires” when a linear combination of its inputs exceeds some (hard or soft) threshold that is, it implements a linear classifier of the kind described in the preceding section.



Artificial Neural Structure

- Neural networks are composed of **nodes** or units connected by directed **links**.
- A link from unit i to unit j serves to propagate the activation a_i from i to j .
- Each link also has a numeric weight $w_{i,j}$ associated with it, which determines the strength and sign of the connection.

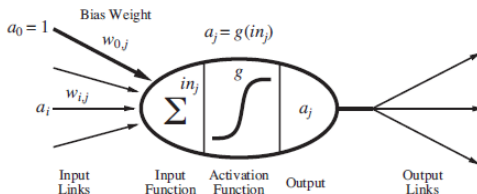
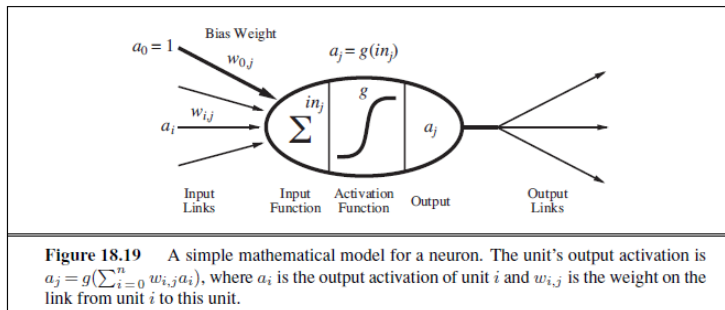


Figure 18.19 A simple mathematical model for a neuron. The unit's output activation is $a_j = g(\sum_{i=0}^n w_{i,j} a_i)$, where a_i is the output activation of unit i and $w_{i,j}$ is the weight on the link from unit i to this unit.



Artificial Neural Structure

- Just as in linear regression models, each unit has a dummy input $a_0 = 1$ with an associated weight $w_{0,j}$.
- Each unit j first computes a weighted sum of its inputs:
 $in_j = \sum_{i=0}^n w_{i,j} a_i$.
- Then it applies an activation function g to this sum to derive the output: $a_j = g(in_j) = g(\sum_{i=0}^n w_{i,j} a_i)$.



Artificial Neural Structure

- The activation function g is typically either a hard threshold, in which case the unit is called a **perceptron**, or a logistic function, in which case the term **sigmoid perceptron** is sometimes used.
- Both of these nonlinear activation function ensure the important property that the entire network of units can represent a nonlinear function.
- The next task is to connect neurons together to form a network.
- There are two fundamentally distinct ways to do this:
 - (i) **feed-forward network**
 - (ii) **recurrent network**
- A feed-forward network has connections only in one direction—that is, it forms a directed acyclic graph.
- A recurrent, on the other hand, feeds its outputs back into its own inputs.



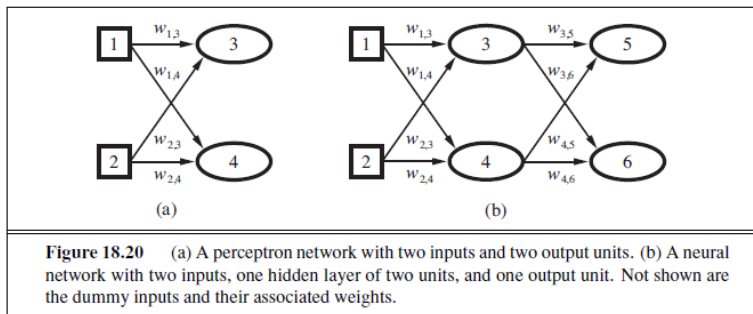
Artificial Neural Structure

- Feed-forward networks are usually arranged in layers, such that each unit receives input only from units in the immediately preceding layer.
- In the next two subsections, we will look at **single-layer networks**, in which every unit connects directly from the network's inputs to its outputs, and **multilayer networks**, which have one or more layers of **hidden units** that are not connected to the outputs of the network.



Single-layer feed-forward neural networks (perceptrons)

- A network with all the inputs connected directly to the outputs is called a **single-layer neural network, or a perceptron network**.
- Figure shows a simple two-input, two-output perceptron network.



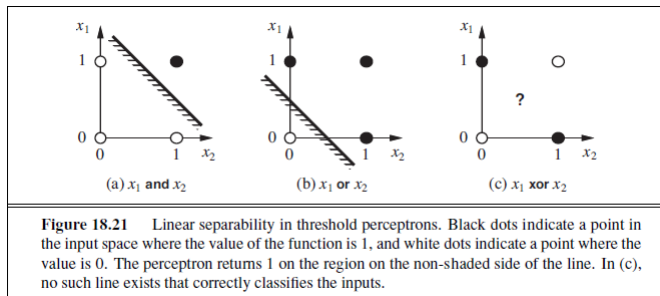
Single-layer feed-forward neural networks (perceptrons)

- The first thing to notice is that a perceptron network with m outputs is really m separate networks, because each weight affects only one of the outputs. Thus, there will be m separate training processes.
- Furthermore, depending on the type of activation function used, the training processes will be either **the perceptron learning rule** or **gradient descent rule for the logistic regression**.
- If you try either method on the two-bit-adder data, something interesting happens. Unit 3 learns the carry function easily, but unit 4 completely fails to learn the sum function.

x_1	x_2	y_3 (carry)	y_4 (sum)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



Single-layer feed-forward neural networks (perceptrons)



- We saw that linear classifiers (whether hard or soft) can represent linear decision boundaries in the input space.
- This works fine for the carry function, which is a logical AND
- The sum function, however, is an XOR (exclusive OR) of the two inputs. This function is not linearly separable so the perceptron cannot learn it.



Exercise from perceptrons

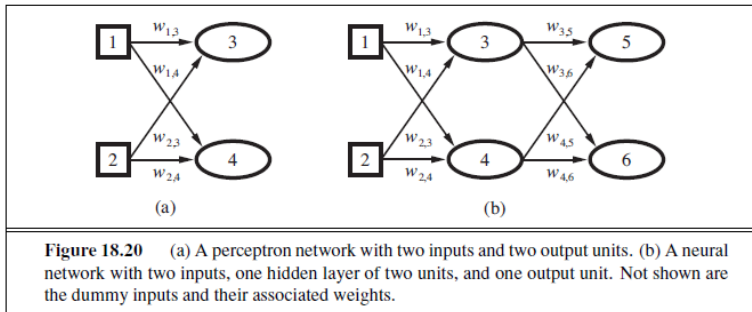
- Train a perceptron to implement the logical AND function using the following data. Assume: Initial weights $w_1 = 0$, $w_2 = 0$, bias $b = 0$, and learning rate $\eta = 1$.

x_1	x_2	Target t
0	0	0
0	1	0
1	0	0
1	1	1



Multilayer feed-forward neural networks

- Consider the simple network shown in Figure (b), which has two input units, two hidden units, and two output unit. (In addition, each unit has a dummy input fixed at 1.)



Multilayer feed-forward neural networks

- Given an input vector $x = (x_1, x_2)$, the activations of the input units are set to $(a_1, a_2) = (x_1, x_2)$. The output at unit 5 is given by

$$\begin{aligned}a_5 &= g(w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4) \\&= g(w_{0,5} + w_{3,5}g(w_{0,3} + w_{1,3}a_1 + w_{2,3}a_2) + w_{4,5}g(w_{0,4} + w_{1,4}a_1 + w_{2,4}a_2)) \\&= g(w_{0,5} + w_{3,5}g(w_{0,3} + w_{1,3}x_1 + w_{2,3}x_2) + w_{4,5}g(w_{0,4} + w_{1,4}x_1 + w_{2,4}x_2))\end{aligned}$$

- Thus, we have the output expressed as a function of the inputs and the weights. A similar expression holds for unit 6.



Multilayer feed-forward neural networks

- As long as we can calculate the derivatives of such expressions with respect to the weights, we can use the gradient-descent loss-minimization method to train the network.
- Because the function represented by a network can be highly nonlinear—composed, as it is, of nested nonlinear soft threshold functions—we can see neural networks as a tool for doing nonlinear regression.



Learning in multilayer networks

- When the network has multiple outputs, we should think of the network as implementing a vector function h_w rather than a scalar function h_w ;
- The network in Figure (b) returns a vector $[a_5, a_6]$. Similarly, the target output will be a vector y .
- Whereas a perceptron network decomposes into m separate learning problems for an m -output problem, this decomposition fails in a multilayer network.
- For example, both a_5 and a_6 in Figure (b) depend on all of the input-layer weights, so updates to those weights will depend on errors in both a_5 and a_6 .



Learning in multilayer networks

- Fortunately, this dependency is very simple in the case of any loss function that is additive across the components of the error vector $y - h_w(x)$.
- For the L_2 loss, we have, for any weight w ,
$$\frac{\partial}{\partial w} \text{Loss}(w) = \frac{\partial}{\partial w} |y - h_w(x)|^2 = \frac{\partial}{\partial w} \sum_k (y_k - a_k)^2 = \sum_k \frac{\partial}{\partial w} (y_k - a_k)^2$$
 where the index k ranges over nodes in the output layer.
- Each term in the final summation is just the gradient of the loss for the k th output.
- Hence, we can decompose an m -output learning problem into m learning problems.
- The major complication comes from the addition of hidden layers to the network. Whereas the error $y - h_w$ at the output layer is clear.



Back-propagation

- the error at the hidden layers seems mysterious because the training data do not say what value the hidden nodes should have.
- Fortunately, it turns out that we can back-propagate the error from the output layer to the hidden layers.
- The back-propagation process emerges directly from a derivation of the overall error gradient.
- At the output layer, the weight-update rule is identical to $w_i \leftarrow w_i + \alpha(y - h_w(x)) * h_w(x)(1 - h_w(x)) * x_i$.
- We have multiple output units, so let Err_k be the k th component of the error vector $y - h_w$.



Backpropagation

- We will also find it convenient to define a modified error $\nabla_k = Err_k * g'(in_k)$, so that the weight update rule becomes $w_{j,k} \leftarrow w_{j,k} + \alpha * a_j * \nabla_k$.
- To update the connections between the input units and the hidden units, we need to define a quantity analogous to the error term for output nodes.
- Here is where we do the error backpropagation.



Backpropagation

- The idea is that hidden node j is “responsible” for some fraction of the error ∇_k in each of the output nodes to which it connects.
- Thus, the ∇_k values are divided according to the strength of the connection between the hidden node and the output node and are propagated back to provide the ∇_j values for the hidden layer.
- The propagation rule for the ∇_j values is the following:
$$\nabla_j = g'(in_j) \sum_K w_{j,k} \nabla_k.$$
- Now the weight-update rule for the weights between the inputs and the hidden layer is essentially identical to the update rule for the output layer: $w_{i,j} \leftarrow w_{i,j} + \alpha * a_i * \nabla_j$.



Backpropagation Algorithm

```
function BACK-PROP-LEARNING(examples, network) returns a neural network
  inputs: examples, a set of examples, each with input vector  $\mathbf{x}$  and output vector  $\mathbf{y}$ 
           network, a multilayer network with  $L$  layers, weights  $w_{i,j}$ , activation function  $g$ 
  local variables:  $\Delta$ , a vector of errors, indexed by network node

  repeat
    for each weight  $w_{i,j}$  in network do
       $w_{i,j} \leftarrow$  a small random number
    for each example  $(\mathbf{x}, \mathbf{y})$  in examples do
      /* Propagate the inputs forward to compute the outputs */
      for each node  $i$  in the input layer do
         $a_i \leftarrow x_i$ 
      for  $\ell = 2$  to  $L$  do
        for each node  $j$  in layer  $\ell$  do
           $in_j \leftarrow \sum_i w_{i,j} a_i$ 
           $a_j \leftarrow g(in_j)$ 
      /* Propagate deltas backward from output layer to input layer */
      for each node  $j$  in the output layer do
         $\Delta[j] \leftarrow g'(in_j) \times (y_j - a_j)$ 
      for  $\ell = L - 1$  to  $1$  do
        for each node  $i$  in layer  $\ell$  do
           $\Delta[i] \leftarrow g'(in_i) \sum_j w_{i,j} \Delta[j]$ 
      /* Update every weight in network using deltas */
      for each weight  $w_{i,j}$  in network do
         $w_{i,j} \leftarrow w_{i,j} + \alpha \times a_i \times \Delta[j]$ 
  until some stopping criterion is satisfied
  return network
```



Backpropagation summarization

The back-propagation process can be summarized as follows

- Compute the ∇ values for the output units, using the observed error.
- Starting with output layer, repeat the following for each layer in the network, until the earliest hidden layer is reached:
- Propagate the ∇ values back to the previous layer.
- Update the weights between the two layers.



Gradient hidden to Output Layer

- we compute just the gradient for $Loss_k = (y_k - a_k)^2$ at the k_{th} output. The gradient of this loss with respect to weights connecting the hidden layer to the output layer will be zero except for weights $w_{j,k}$ that connect to the k_{th} output unit.

$$\begin{aligned}\frac{\partial Loss_k}{\partial w_{j,k}} &= -2(y_k - a_k) \frac{\partial a_k}{\partial w_{j,k}} = -2(y_k - a_k) \frac{\partial g(in_k)}{\partial w_{j,k}} \\ &= -2(y_k - a_k) g'(in_k) \frac{\partial in_k}{\partial w_{j,k}} = -2(y_k - a_k) g'(in_k) \frac{\partial}{\partial w_{j,k}} \left(\sum_j w_{j,k} a_j \right) \\ &= -2(y_k - a_k) g'(in_k) a_j = -a_j \Delta_k ,\end{aligned}$$



Gradient input Layer to hidden Layer

- To obtain the gradient with respect to the $w_{i,j}$ weights connecting the input layer to the hidden layer, we have to expand out the activations a_j and reapply the chain rule. We will show the derivation in gory detail because it is interesting to see how the derivative operator propagates back through the network:

$$\begin{aligned}\frac{\partial Loss_k}{\partial w_{i,j}} &= -2(y_k - a_k) \frac{\partial a_k}{\partial w_{i,j}} = -2(y_k - a_k) \frac{\partial g(in_k)}{\partial w_{i,j}} \\&= -2(y_k - a_k) g'(in_k) \frac{\partial in_k}{\partial w_{i,j}} = -2\Delta_k \frac{\partial}{\partial w_{i,j}} \left(\sum_j w_{j,k} a_j \right) \\&= -2\Delta_k w_{j,k} \frac{\partial a_j}{\partial w_{i,j}} = -2\Delta_k w_{j,k} \frac{\partial g(in_j)}{\partial w_{i,j}} \\&= -2\Delta_k w_{j,k} g'(in_j) \frac{\partial in_j}{\partial w_{i,j}} \\&= -2\Delta_k w_{j,k} g'(in_j) \frac{\partial}{\partial w_{i,j}} \left(\sum_i w_{i,j} a_i \right) \\&= -2\Delta_k w_{j,k} g'(in_j) a_i = -a_i \Delta_j ,\end{aligned}$$



Table of Contents

- 1 Forms of Learning
- 2 Supervised Learning
- 3 Regression and Classification with Linear Models
 - Univariate linear regression
 - Multivariate linear regression
 - Linear classifiers with a hard threshold
 - Linear classification with logistic regression
- 4 Artificial Neural Networks
 - Artificial Neural Structure
 - Single-layer feed-forward neural networks (perceptrons)
 - Multilayer feed-forward neural networks
 - Learning in multilayer networks
- 5 References



References

-  Stuart Russell and Peter Norvig. 2009. Artificial Intelligence: A Modern Approach (3rd ed.). Prentice Hall Press, Upper Saddle River, NJ, USA.
-  <https://www.cs.cmu.edu/~15281/coursenotes/probability>
-  <https://www.cs.cmu.edu/~15281/coursenotes/bayesnets>
-  https://www.cs.cmu.edu/~15281/coursenotes/hmm_pf

